

JavaScript Methods Cheat Sheet

call / apply / bind · Array · Object · String · Promise — Description · Use Case · Example

■ FUNCTION METHODS — call / apply / bind

.call(thisArg, ...args)

Invokes fn immediately with explicit this + individual args.

■ Borrow methods, set this on the fly

```
const u = { name: 'Akshay' };  
function greet(c){return `${this.name}, ${c}`;}  
greet.call(u, 'Pune'); // 'Akshay, Pune'
```

.apply(thisArg, [args])

Like .call() but args passed as an ARRAY.

■ Math.max on array, spread args

```
greet.apply(u, ['Mumbai']); // 'Akshay, Mumbai'  
Math.max.apply(null, [1, 5, 3]); // 5
```

.bind(thisArg, ...args)

Returns NEW function with this permanently bound. Does NOT call immediately.

■ Event handlers, partial application, React class methods

```
const fn = greet.bind(u, 'Pune');  
fn(); // 'Akshay, Pune' — called later  
const add5 = add.bind(null, 5); add5(3); // 8
```

■ ARRAY (1/2) map filter reduce forEach find some flat

.map(fn)

NEW array — each element transformed. Same length.

■ Transform data, render lists in React

```
[1, 2, 3].map(x=>x*2); // [2, 4, 6]  
users.map(u=>u.name); // ['A', 'B']
```

.filter(fn)

NEW array — only elements where fn returns true.

■ Search results, remove items, conditional lists

```
[1, 2, 3, 4].filter(x=>x>2); // [3, 4]  
users.filter(u=>u.active);
```

.reduce(fn, init)

Reduces array to a SINGLE value by accumulating.

■ Sum, count, group, build objects from arrays

```
[1, 2, 3, 4].reduce((a, x)=>a+x, 0); // 10  
// group by role, build lookup maps
```

.forEach(fn)

Iterates array. Returns undefined. Cannot break early.

■ Side effects: logging, DOM updates. Not for transforms.

```
[1, 2, 3].forEach(x=>console.log(x));  
// returns undefined — cannot chain!
```

.find(fn) / .findIndex(fn)

find()→first matching element. findIndex()→its index. Both stop early.

■ Look up item by id, check position

```
users.find(u=>u.id===3); // {id:3}  
users.findIndex(u=>u.id===3); // 2
```

.some(fn) / .every(fn)

some()→true if ANY passes. every()→true if ALL pass. Short-circuit.

■ Validate fields, permission checks

```
[1, 2, 3].some(x=>x>2); // true  
[1, 2, 3].every(x=>x>0); // true
```

.flat(depth) / .flatMap(fn)

flat()→flatten nested arrays. flatMap()→map then flatten by 1.

■ Flatten API responses, expand mapped arrays

```
[[1, 2], [3, 4]].flat(); // [1, 2, 3, 4]  
[1, 2].flatMap(x=>[x, x*2]); // [1, 2, 2, 4]
```

■ ARRAY (2/2) sort reverse slice splice includes at fill

.sort(fn)

Mutates array in place. Always pass comparator for numbers!

■ Sort tables, order lists

```
[3, 1, 2].sort((a, b)=>a-b); // [1, 2, 3] asc  
[3, 1, 2].sort((a, b)=>b-a); // [3, 2, 1] desc
```

.reverse()

Reverses array IN PLACE — mutates original.

■ Reverse chronological order, flip arrays

```
[1, 2, 3].reverse(); // [3, 2, 1]  
// Use [...arr].reverse() to avoid mutation
```

.slice(start, end)

Returns NEW sub-array. Non-mutating. end is exclusive.

■ Pagination, copy array, take first N items

```
[1, 2, 3, 4, 5].slice(1, 3); // [2, 3]  
arr.slice(); // shallow copy
```

.splice(start, del, ...items)

MUTATES — remove/insert/replace elements in place.

■ Remove by index, insert at position

```
const a=[1,2,3,4];  
a.splice(1,2); // removes 2 -> [1,4]  
a.splice(1,0,9); // insert 9 at index 1
```

.includes(val) / .indexOf(val)

includes()→true/false. indexOf()→index or -1. includes handles NaN.

■ Check membership, find position

```
[1, 2, 3].includes(2); // true  
[1, 2, 3].indexOf(2); // 1  
[NaN].includes(NaN); // true (indexOf fails)
```

.at(index)

Access element by index. Accepts NEGATIVE indices from end.

■ Get last element without arr.length-1

```
[1, 2, 3].at(0); // 1 — first  
[1, 2, 3].at(-1); // 3 — last  
[1, 2, 3].at(-2); // 2 — second to last
```

.fill(val, start, end)

Fills elements with a static value. Mutates original.

■ Initialize arrays, reset values, test data

```
new Array(3).fill(0); // [0, 0, 0]  
[1, 2, 3, 4].fill(0, 1, 3); // [1, 0, 0, 4]
```

■ OBJECT (1/2) keys values entries assign freeze

Object.keys(obj)

Array of own enumerable PROPERTY NAMES.

■ Iterate keys, count properties

```
Object.keys({a:1,b:2}); // ['a','b']
Object.keys(u).length; // count props
```

Object.values(obj)

Array of own enumerable VALUES.

■ Sum values, render value list

```
Object.values({a:1,b:2}); // [1,2]
Object.values(s).reduce((a,v)=>a+v,0);
```

Object.entries(obj)

Array of [key, value] pairs. Pairs with Map constructor.

■ Iterate key+value, convert to Map

```
Object.entries({a:1}); // [['a',1]]
for(const [k,v] of Object.entries(obj)){}
```

Object.assign(target,...src c)

Copies own enumerable props from sources to target. Shallow. Mutates target.

■ Merge configs, shallow clone

```
Object.assign({},defaults,overrides);
const copy=Object.assign({},original);
```

Object.freeze(obj)

IMMUTABLE — no add/edit/delete. Shallow freeze only.

■ Constants, config objects, prevent mutations

```
const c=Object.freeze({api:'/v1'});
c.api='/v2'; // silently ignored
```

■ OBJECT (2/2) seal create hasOwn fromEntries getPrototypeOf

Object.seal(obj)

Prevents ADD/DELETE. Allows editing existing props.

■ Fixed-shape objects that need value updates

```
const o=Object.seal({name:'Akshay'});
o.name='Rahul'; // OK — edit allowed
o.age=25; // silently ignored
```

Object.create(proto)

Creates new object with specified prototype.

■ Prototypal inheritance, custom prototype chain

```
const animal={speak(){return 'sound'}};
const dog=Object.create(animal);
dog.speak(); // 'sound' — inherited
```

Object.hasOwn(obj, key)

Checks own property (not inherited). Replaces hasOwnProperty.

■ Safe property check, avoid prototype pollution

```
Object.hasOwn({a:1},'a'); // true
Object.hasOwn({a:1},'toString');// false
```

Object.fromEntries(iter)

Inverse of Object.entries() — [key,val] pairs → object.

■ Transform objects via map/filter, Map → object

```
Object.fromEntries([[a,1]]); // {a:1}
// filter: Object.fromEntries(
// Object.entries(o).filter(([k,v])=>v>0))
```

Object.getPrototypeOf(obj)

Returns the prototype. Read-only inspection.

■ Debug inheritance, check extends

```
Object.getPrototypeOf([])
=== Array.prototype; // true
```

Aa STRING trim pad replaceAll startsWith includes at

.trim() / .trimStart() / .trimEnd()

Remove whitespace from both/start/end.

■ Sanitize form input, clean API data

```
' hello '.trim(); // 'hello'
' hello '.trimStart(); // 'hello '
```

.padStart(len,ch) / .padEnd(len,ch)

Pad string to target length from start or end.

■ Format IDs, time display (01:05), align output

```
'5'.padStart(3,'0'); // '005'
'hi'.padEnd(5,'. '); // 'hi... '
```

.replaceAll(search, replace)

Replaces ALL occurrences. replace() only replaces first.

■ Clean strings, template substitution

```
'a-b-c'.replaceAll('-', '_'); // 'a_b_c'
'hello'.replace('l','L'); // 'heLlo'
```

.startsWith(s) / .endsWith(s)

Check if string starts or ends with value. Case-sensitive.

■ URL routing, file type checks, prefix validation

```
'hello world'.startsWith('hello'); // true
'image.png'.endsWith('.png'); // true
```

.includes(str) / .at(index)

includes()→substring check. at()→char by index (supports negative).

■ Search in strings, get last character

```
'hello world'.includes('world'); // true
'hello'.at(-1); // 'o'
```

■ PROMISE all allSettled race any resolve reject

Promise.all(arr)

ALL must resolve. Returns array of results. FAILS FAST on first rejection.

■ Parallel calls where all results needed

```
const [user,posts]=await Promise.all([
  fetchUser(1), fetchPosts(1)
]); // total = slowest call
```

Promise.allSettled(arr)

Waits for ALL regardless of outcome. Never rejects itself.

■ Need all results even if some fail

```
const res=await Promise.allSettled([p1,p2]);
res.forEach(r=>{
  if(r.status==='fulfilled') use(r.value);
```

Promise.race(arr)

First promise to SETTLE (resolve or reject) wins.

■ Timeout pattern — race fetch vs timeout promise

```
const timeout=new Promise( (_,rej)=>
  setTimeout(()=>rej('Timeout'),3000));
await Promise.race([fetchData(),timeout]);
```

Promise.any(arr)

First to RESOLVE wins. Ignores rejections unless ALL reject.

■ Try multiple sources, use fastest success

```
await Promise.any([mirror1,mirror2,m3]);
// Throws AggregateError only if ALL reject
```

Promise.resolve(v) / .reject(r)

Create already-resolved or already-rejected promise.

■ Return cached data, testing, wrap sync values

```
Promise.resolve(42).then(v=>console.log(v));
Promise.reject(new Error('fail')).catch(e=>handle(e));
```

★ QUICK MEMORY — KEY DISTINCTIONS

call vs apply vs bind	call=immediate+spread args apply=immediate+array bind=returns new fn, called later
map vs forEach	map=returns NEW array (chain it) forEach=returns undefined (side effects only)
slice vs splice	slice=non-mutating, returns copy splice=mutates original, remove/insert in place
freeze vs seal	freeze=no add/edit/delete (fully locked) seal=no add/delete but can edit values
Promise.all vs .any	all=ALL must resolve, fails fast any=FIRST resolve wins, ignores rejections
find vs filter	find=returns first match (stops early) filter=returns ALL matches as new array